

Event Bubbling in JavaScript

Event Bubbling ka matlab hai:

Jab kisi **child element** par event hota hai, to event **child** → **parent** → **grandparent** → **document** ki taraf upar propagate karta hai.

Example

HTML

```
<div id="parent">
  <button id="child">Click Me</button>
</div>

<script>
const parent = document.querySelector("#parent");
const child = document.querySelector("#child");

parent.addEventListener("click", () => {
  console.log("Parent clicked");
});
```

</>

▶

📄

```
child.addEventListener("click", () => {  
    console.log("Child clicked");  
});  
</script>
```

Agar **button** par click karoge:

```
Child clicked  
Parent clicked
```

Flow

```
<button>      ← Event yaha hua  
  ↑  
<div>         ← Event yaha bubble hua  
  ↑  
<body>  
  ↑  
<html>  
  ↑  
document
```



`event.target` vs `event.currentTarget`

Ye interview mein bahut important hai:

`</>` **JavaScript**

```
parent.addEventListener("click", (event) => {  
    console.log("target:", event.target);  
    console.log("currentTarget:", event.currentTarget);  
});
```

Agar button par click hua:

`event.target`
→ button

`event.currentTarget`
→ parent div

`target` = jis element par actually event hua

`currentTarget` = jis element par listener laga hua hai

...

Event Delegation ka matlab hai:

Har child element par alag event listener lagane ke bajay, **parent element par ek hi event listener** lagana aur `event.target` se pata lagana ki kis child par event hua.

Simple Example

HTML

```
<ul id="users">
  <li>Tarun</li>
  <li>Rahul</li>
  <li>Aman</li>
</ul>
```

✗ Har `<li>` par listener lagane ki zarurat nahi:

✗ Har `<li>` par listener lagane ki zarurat nahi:

JavaScript

```
document.querySelector("#users").addEventListener("click", (event) => {

  if (event.target.tagName === "LI") {
    console.log("Clicked:", event.target.textContent);
  }

});
```

Agar **Rahul** par click kiya:

Clicked: Rahul

Ye kaise kaam karta hai?

```

click
  ↓
<li> Rahul
  ↓
Event Bubbling
  ↓
<ul> users
  ↓
Parent ka listener
  ↓
event.target
  ↓
<li> Rahul
  
```



Yahan **event bubbling** important hai. Child ka event bubble hokar parent tak aata hai, aur parent `event.target` se actual clicked child identify karta hai.

Dynamic Elements mein bahut useful

Maan lo initially:

HTML

```

<ul id="users">
  <li>Tarun</li>
  <li>Rahul</li>
</ul>
  
```

`</>` ▶

Baad mein JavaScript se naya `<li>` add kiya:

JavaScript

```
users.innerHTML += "<li>Aman</li>";
```



Event delegation mein **Aman par bhi listener automatically kaam karega**, kyunki listener parent `<ul>` par laga hai.

Bubbling ko rokna

Agar child ka event parent tak nahi jana chahiye:

`</>` JavaScript

```
child.addEventListener("click", (event) => {  
    event.stopPropagation();  
  
    console.log("Child clicked");  
});
```

Ab output:

Child clicked

Parent clicked nahi chalega.

Debouncing in JavaScript

Debouncing ka matlab hai:

Kisi function ko **turant baar-baar execute karne ke bajay**, tab execute karna jab user ne kuch time ke liye action karna **stop** kar diya ho.

Real-life Example: Search Box

Maan lo user search box mein type karta hai:

```
J  
Ja  
Jav  
Java  
JavaS  
JavaSc  
JavaScr
```



Agar har keypress par API call karein, to **7 API calls** ho sakti hain.

Debouncing mein hum bolte hain:

"User typing stop kare, phir 500ms baad API call karo."

JavaScript

```
let timer;

function search(value) {
  clearTimeout(timer);

  timer = setTimeout(() => {
    console.log("API call:", value);
  }, 500);
}
```

HTML

```
<input type="text" oninput="search(this.value)">
```

Agar user continuously type kar raha hai:

```
J → timer
Ja → timer reset
Jav → timer reset
Java → timer reset
```

Jab user 500ms tak type nahi karega, tab:

API call: Java

Event Bubbling vs Event Delegation

Event Bubbling

Event ka propagation mechanism hai

Child → Parent jata hai

Automatically hota hai

`stopPropagation()` se roka ja sakta hai

Event Delegation

Event handle karne ki technique hai

Parent par listener lagate hain

Developer intentionally use karta hai

Usually `event.target` use hota hai

Throttling in JavaScript

Throttling ka matlab hai:

Kisi function ko **fixed time interval mein maximum ek baar** execute karna, chahe event kitni bhi baar trigger ho.

Simple Example

Maan lo `scroll` event bahut baar fire ho raha hai:

</> JavaScript

```
window.addEventListener("scroll", () => {  
  console.log("Scrolling...");  
});
```



Scroll karte waqt ye function **bahut baar** execute ho sakta hai.

Throttling lagane par:

```
</> JavaScript

let allow = true;

window.addEventListener("scroll", () => {
  if (!allow) return;

  console.log("Scrolling...");

  allow = false;

  setTimeout(() => {
    allow = true;
  }, 1000);
});
```

Ab function **maximum 1 baar per second** execute hoga.

Flow

Scroll → Execute ✓
Scroll → Ignore ✗
Scroll → Ignore ✗
Scroll → Ignore ✗
1 second complete
Scroll → Execute ✓



Debouncing vs Throttling

Debouncing

User ke rukne ke baad execute

Continuous events mein last event important

Search box ke liye useful

500ms wait after last event

Throttling

Fixed interval par execute

Continuous events mein regular execution

Scroll / mousemove ke liye useful

500ms mein max 1 execution



Memoization kya hai?

Memoization ek optimization technique hai jisme hum kisi function ke **previous result** ko **cache/store** kar lete hain.

Agar same input dobara aata hai, to function ko dobara calculate karne ke bajay **stored result** return kar dete hain.

Simple Example

Normal function:

JavaScript

```
function square(n) {  
  console.log("Calculation...");  
  return n * n;  
}  
  
console.log(square(5));  
console.log(square(5));
```



Output:

Calculation...

25

Calculation...

25

5 ka square dono baar calculate hua.

Memoization ke saath

</> JavaScript

```
function memoize(fn) {  
  const cache = {};  
  
  return function (n) {  
    if (cache[n] !== undefined) {  
      console.log("Cache se result");  
      return cache[n];  
    }  
  }  
}
```



</> JavaScript

```
const cache = {};  
  
return function (n) {  
  if (cache[n] !== undefined) {  
    console.log("Cache se result");  
    return cache[n];  
  }  
  
  console.log("Calculation...");  
  const result = fn(n);  
  
  cache[n] = result;  
  
  return result;  
};  
}  
  
const square = memoize((n) => n * n);  
  
console.log(square(5));  
console.log(square(5));  
console.log(square(10));  
console.log(square(5));
```



Output:

Calculation...

25

```
Cache se result
```

25

Calculation...

100

```
Cache se result
```

25

Flow

```
square(5)
```


Cache mein hai? 

Calculate $\rightarrow 25$

↓

Cache mein store

↓
Calculate → 25
↓
Cache mein store

square(5)
↓
Cache mein hai? ☒
↓
Direct 25 return

Kahan useful hai?

Use Case	Benefit	📄
Expensive calculations	Calculation repeat nahi hoti	
Fibonacci	Recursive calls kam	
API/data processing	Same result reuse	
React	Unnecessary calculations avoid karne mein useful	
Dynamic Programming	Previous results store karna	

Part 1 — `call()`, `apply()`, `bind()`

1. Sabse pehle `this` samjho

`call()`, `apply()` aur `bind()` ka main purpose samajhne ke liye `this` samajhna zaroori hai.

 JavaScript 

```
const user = {  
  name: "Tarun"  
};  
  
function showName() {  
  console.log(this.name);  
}  
  
showName.call(user);
```

Output:

Tarun



```
// JavaScript
```

```
this = user
```

Matlab `call()` ki help se hum decide kar sakte hain ki function ke andar `this` kis object ko refer karega.

2. `call()` kya hai?

Simple definition:

`call()` kisi function ko **immediately execute** karta hai aur us function ke andar `this` ko manually set karta hai.

Syntax:

```
// JavaScript
```

```
function.call(thisObject, arg1, arg2, arg3)
```

```
const user = {  
  name: "Tarun"  
};  
  
function introduce(age, city) {  
  console.log(this.name);  
  console.log(age);  
  console.log(city);  
}  
  
introduce.call(user, 22, "Indore");
```

Output:

```
Tarun  
22  
Indore
```

Yahan:

Yahan:

```
</> JavaScript
```

```
introduce.call(user, 22, "Indore");
```

ka matlab:

```
this = user
```

```
age = 22
```

```
city = "Indore"
```

3. Normal function vs `call()`

Normal:

```
</> JavaScript

function greet() {
  console.log(this.name);
}

const user = {
  name: "Tarun"
};

greet();
```

`this` ka value context par depend karega.

Lekin:

```
</> JavaScript

greet.call(user);
```



Ab clearly:

this → user

4. `call()` multiple arguments

JavaScript

```
const person = {  
  name: "Rahul"  
};  
  
function intro(age, profession) {  
  console.log(  
    this.name,  
    age,  
    profession  
  );  
}  
  
intro.call(person, 22, "Developer");
```



Output:

Rahul 22 Developer

Important:

`call()` me arguments **comma-separated** pass hote hain:

`</>` JavaScript

```
call(object, arg1, arg2, arg3)
```

5. `apply()` kya hai?

`apply()` almost `call()` jaisa hi hai.

Difference mainly arguments pass karne ka hai.

`call()` :



Difference mainly **arguments** pass karne ka hai.

call() :

</> JavaScript

```
function intro(age, city) {  
    console.log(this.name, age, city);  
}  
  
intro.call(user, 22, "Indore");
```

apply() :

</> JavaScript

```
intro.apply(user, [22, "Indore"]);
```

Output dono ka same:

Tarun 22 Indore



★ 6. `call()` vs `apply()`

`call()`

Function immediately execute

`this` set kar sakte hain

Arguments individually

`call(obj, a, b)`

`apply()`

Function immediately execute

`this` set kar sakte hain

Arguments array/array-like me

`apply(obj, [a, b])`

Yaad rakhne ka shortcut:

CALL → Comma

APPLY → Array

Example:

JavaScript

`call(user, 10, 20)`

VS

JavaScript

`apply(user, [10, 20])`

7. `bind()` kya hai?

Yahan important difference aata hai.

`bind()` function ko **immediately execute nahi karta**.

Ye ek **new function return karta hai**, jisme `this` aur optionally arguments permanently bind ho jaate hain.

Example:

```
</> JavaScript

const user = {
  name: "Tarun"
};

function greet() {
  console.log(this.name);
}

const newFunction = greet.bind(user);

newFunction();
```

-9cc7-46834056b199

JavaScript

Output:

Tarun

Flow:

```
greet()  
  ↓  
bind(user)  
  ↓  
new function  
  ↓  
newFunction()  
  ↓  
execute
```

★ 8. `call()` vs `bind()`

`call()`:

```
JavaScript  
greet.call(user);
```

Immediately execute.

`bind()`:

```
JavaScript  
const fn = greet.bind(user);  
  
fn();
```

Pehle new function create hota hai.

Baad me execute karte hain.

★ 10. Most Important Interview Table

Feature	<code>call()</code>	<code>apply()</code>	<code>bind()</code>
<code>this</code> set karta hai	✓	✓	✓
Immediately execute	✓	✓	✗
New function return	✗	✗	✓
Arguments	Individual	Array	Individual
Syntax	<code>call(obj,a,b)</code>	<code>apply(obj,[a,b])</code>	<code>bind(obj,a,b)</code>

11. Real-world example

Maan lo:

JavaScript

```
const user1 = {
  name: "Tarun"
};

const user2 = {
  name: "Rahul"
};

function sayHello() {
  console.log("Hello " + this.name);
}
```

Same function ko different objects ke saath chala sakte ho:

JavaScript

```
sayHello.call(user1);
sayHello.call(user2);
```



12. Function Borrowing

Ye interview me poocha ja sakta hai.

Maan lo:

```
JavaScript

const person1 = {
  name: "Tarun",

  greet() {
    console.log("Hello " + this.name);
  }
};

const person2 = {
  name: "Rahul"
};
```

`person2` ke paas `greet()` method nahi hai.

Lekin:



13. bind() real-world example

Suppose:

```
</> JavaScript

const user = {
  name: "Tarun",

  greet() {
    console.log(this.name);
  }
};
```

Agar method ko variable me store kar diya:

```
</> JavaScript

const greet = user.greet;

greet();
```

To `this` lose ho sakta hai.



To `this` lose ho sakta hai.

Bind:

```
</> JavaScript  
  
const greet = user.greet.bind(user);  
  
greet();
```

Output:

Tarun

14. Interview Question

Q: `call`, `apply`, `bind` kyu use karte hain?

Answer:

Inka use function ke `this` context ko explicitly control karne ke liye hota hai. `call()` aur `apply()` function ko immediately execute karte hain, jabki `bind()` ek new function return karta hai. `call()` arguments individually leta hai, `apply()` array ke form me arguments leta hai.

Ye answer interview me bol sakte ho. 

`person2` ke paas `greet()` method nahi hai.

Lekin:

`</>` **JavaScript**

```
person1.greet.call(person2);
```

Output:

Hello Rahul

Isko kehte hain:

Function Borrowing

Ek object ka function temporarily doosre object ke liye use karna.